Can This Agent Even Do That? A Decidable Goal-Achievability Type Discipline for LLM-Synthesized Agent Skills

Anonymous Author(s)
Affiliation
Address
`email`

Abstract

LLM agents are increasingly driven by natural-language specifications, expressed through artifacts such as `SKILL.md` and `AGENT.md` files that describe an agent's capabilities, tools, and workflows. Agent Skills, for example, package instructions that guide an agent in performing specific tasks [5]. But how can we know whether such specifications are themselves correct? Even if an LLM agent follows a specification exactly, can it actually achieve the goal that the specification is intended to accomplish? Or might the goal be fundamentally unachievable, discovered only after spending substantial time and tokens executing the agent—or even ending in deadlock? We present a *skill compiler* that formally validates, before runtime execution and without relying on an LLM judge, whether a skill's *goal is achievable*. The compiler combines capability-guarded reachability from automated planning with deadlock-freedom from simple multiparty sessions (SMPS), decided *directly* over session configurations and a goal-marked global type synthesized from the natural-language specification. Our approach follows the SMPS discipline of checking a session directly against a global protocol [1, 2]. The approach begins by using an LLM to extract the content of a natural-language specification and compile it into a formal logical pack consisting of preconditions, effects, and goals, which are then consumed by the inference rules. The checker is specified by a Coq-verified, refutation-sound abstraction theorem. We further establish subject reduction and session fidelity for the direct discipline. The single-label communication fragment with bystander interleaving is mechanized, while world-changing action interleaving is proved under explicit hypotheses of goal stability and effect commutativity. We formally define the language and its behavior, specify how skills and sessions are checked, and prove that the resulting checking framework has several important correctness properties. We also identify which part of the system remains decidable and show where the boundary of undecidability begins when participants can be added dynamically.

1 Introduction

A modern agent is no longer merely a chatbot, but an autonomous system capable of taking actions beyond text generation—for example, writing code, sending emails, or executing trades. Such behaviour is often guided by natural-language artifacts that specify instructions, workflows, and constraints for the agent [3, 4, 6, 7]. A *skill* is one common way of expressing such guidance. It is a folder containing instructions and tooling that an agent can discover and load as needed [5], thereby *harnessing* a general-purpose agent toward a particular outcome.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

There is growing evidence that this guidance matters. Across seven state-of-the-art open-source multi-agent systems, failure rates range from 41% to 86.7% [8]. More importantly, some of these failures can be addressed without changing the underlying model. In one system, simply adding a task-objective verification step to the specification improves task success by 15.6% [8, Appendix H]. Likewise, repairing the harness based on execution traces improves task completion by 6.3 to 18.4 percentage points across four agent benchmarks [4]. These results show that the instructions and structure surrounding an agent can matter as much as the model itself.

The question, however, is how we can know whether such guidance is sufficient to achieve the outcome it declares. A plan may *book a flight and email a confirmation* even though no email tool is available. It may pursue *a flight under $500* even though the available booking tool can return only premium fares. Or a planner may wait for a worker's result while the worker waits for the planner's answer, with each expecting a message that the other will never send. These are not merely execution failures. In each case, the goal is already unreachable before execution begins.

Such failures are structural and recurring, yet today the only way to discover them is often to run the skill and inspect what went wrong afterwards [4]. By then, the agent may already have spent substantial time and tokens taking actions, and a multi-agent system may already have become stuck waiting for communication that will never occur. A simple static check could, in principle, identify these problems before execution.

This paper asks a deliberately narrow question and answers it with a deterministic formal typing system: **given an LLM skill, can its goal be achieved at all?** We do not, here, verify that every step is safe, nor that the agent behaves correctly on every possible input. Instead, we decide *achievability*. The analysis is tolerant: small details, detours, and payload variation should not trigger a false alarm. At the same time, it is *sound for refutation*: when the compiler concludes that a goal is impossible, that verdict is correct.

1.1 The idea, and the trust boundary

The architecture, shown in Figure 1, has two parts separated by a trust boundary. First, an LLM performs *compaction*: it reads the natural-language skill and turns it into a formal *pack* containing capabilities with preconditions and effects, a goal-marked global protocol, and an initial state. This step is *untrusted*: the LLM may misunderstand the skill or omit relevant details.

A deterministic *checker* then decides achievability using only the resulting pack. The checker is mechanically proved sound: if it returns *impossible*, then the goal is impossible under the formal pack it received. If the LLM omits a constraint or capability from the original skill, the checker cannot recover information that is not present in the pack; it will instead reason about the incomplete pack. Conversely, if the LLM introduces a constraint that was not in the original skill, the checker may report the resulting goal as impossible. Thus, our guarantee is about the checker's verdict with respect to the formal pack, not about whether the pack perfectly represents the original natural-language skill.

This does not make the compaction step a weakness that we simply ignore. The formal pack provides a structured target for the LLM to extract. Rather than asking the LLM to freely invent a formal model, we give it a predefined logical structure: capabilities have preconditions and effects, goals are explicit, and the protocol follows a fixed syntax. This makes compaction a constrained translation task and gives us a concrete artifact that can be inspected before execution.

Crucially, the gap between *what the user meant* and *what was formalized* cannot be eliminated. We instead make this gap explicit at the compaction step, where the resulting pack can be inspected and checked before execution. When the compaction is accurate, the checker can identify goals that are impossible before the agent spends time and tokens executing the skill. When the compaction is inaccurate, the checker still provides a sound verdict for the formal pack it received. The key design idea is therefore to move achievability checking from runtime execution to an explicit, inspectable checkpoint before execution.

1.2 Contributions

1. **A goal-achievability type discipline** (Section 3 and Appendices A–C) that combines capability-guarded reachability from automated planning with deadlock-freedom from multiparty session types. The discipline checks whole session configurations directly against

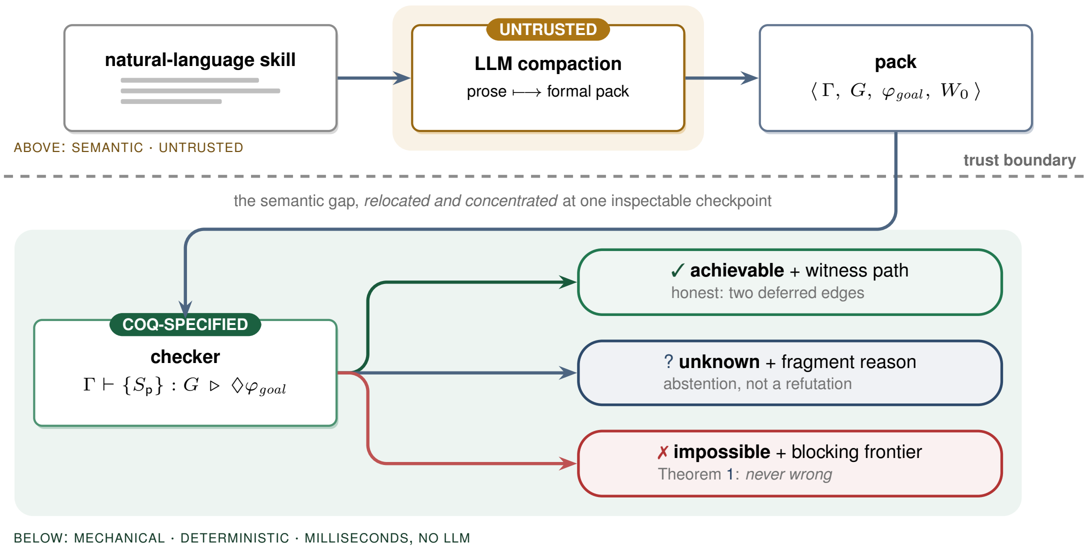

Figure 1: The trust boundary. An untrusted LLM compacts natural-language instructions into a formal pack. A deterministic checker then decides achievability over the pack using a Coq-verified abstraction theorem. The verdicts are asymmetric: *impossible* is sound (Theorem 1), while *achievable* depends on the accuracy of the pack (Section 3.1).

a goal-marked global type synthesized from the skill. Building on the multiparty session type tradition of Honda, Yoshida, and Carbone [12, 13], and the direct session-checking approach of SMPS [1, 2], it adds capabilities, explicit goals, and a refutation-sound but achievement-incomplete analysis for LLM-drafted skills.

2. **A mechanized soundness specification** (Appendix D) formalized in Coq. We prove a refutation-sound abstraction theorem parameterized by simulation hypotheses, together with tolerance soundness and capability monotonicity: adding capabilities cannot make an achievable goal impossible. We also establish operational correspondence through subject reduction and session fidelity for the direct judgment over a labelled transition system. For single-label communication, we mechanize the cases where other participants continue to act concurrently; the world-changing action cases are proved on paper under explicit side conditions.
3. **A decidability boundary** (Appendix D). Achievability is decidable when the set of participants is fixed. We also show that an extension allowing the dynamic addition of an unbounded number of participants becomes undecidable.
4. **An implementation and evaluation** (Sections 4–5). We implement the checker together with an LLM-compaction front-end and a schema gate, and evaluate it on a corpus covering the main failure modes. The results are consistent with the theoretical guarantees: we observe no false impossibility verdicts, while every false achievability verdict falls into one of the two deferred cases.

2 Overview by Example

Consider a one-agent skill whose goal is *a flight is booked and a confirmation is sent*. The LLM compacts the skill into a formal pack. If no email tool is available, there is no action that can establish $\mathsf{confirmation\_sent}$.

$$\begin{aligned} \varphi_{goal} &= \mathsf{booked} \wedge \mathsf{confirmation\_sent} \\ \Gamma(a) &= \langle\, pre_a,\ \langle\, Add_a,\ Del_a,\ Asg_a,\ Nd_a \,\rangle \rangle \end{aligned}$$

$$\begin{array}{lll} \mathsf{search} & \mapsto \langle\, \top, & \langle\, \{\mathsf{searched}\}, \quad \emptyset, \emptyset, \emptyset \,\rangle \rangle \\ \mathsf{filter} & \mapsto \langle\, \mathsf{searched}, & \langle\, \{\mathsf{filtered}\}, \quad \emptyset, \emptyset, \emptyset \,\rangle \rangle \\ \mathsf{book} & \mapsto \langle\, \mathsf{filtered}, & \langle\, \{\mathsf{booked}\}, \quad \emptyset, \emptyset, \emptyset \,\rangle \rangle \\ \mathsf{email} & \mapsto \langle\, \mathsf{booked}, & \langle\, \{\mathsf{confirmation\_sent}\}, \emptyset, \emptyset, \emptyset \,\rangle \rangle \end{array}$$

$$\mathrm{dom}(\Gamma_A) = \{\mathsf{search}, \mathsf{filter}, \mathsf{book}\} \qquad \mathrm{dom}(\Gamma_B) = \mathrm{dom}(\Gamma_A) \cup \{\mathsf{email}\}$$

Under STRIPS [17] frame semantics, the checker therefore returns *impossible* and identifies the missing capability. If the email tool is available, the goal is reachable through the sequence search · filter · book · email, which the checker returns as a witness. The same example is mechanized in Appendix D (the instance `FlightInstance`). Importantly, the booking itself remains reachable in the first case; the refutation concerns only the missing capability needed to complete the goal.

Tolerance is also important. A skill should not be rejected simply because an agent sends extra status messages, takes a clarifying step, or uses a different payload. Our analysis therefore allows these variations in three ways: *existential may-reachability*, where one successful path is sufficient; *session subtyping*, which allows safe interface variation; and *goal-relevant abstraction*, which ignores payload details that do not affect the goal. These mechanisms are defined in Appendices A–C.

3 Verification Model and Signals

Declared pack. The checker consumes $P = \langle \Gamma, G, \varphi_{goal}, W_0 \rangle$. Here, Γ contains capability declarations with guards and STRIPS-style effects; G is a goal-marked global protocol (a *global type*; Appendix A); φ_{goal} is the logical goal; and W_0 is the initial abstract world state. The LLM produces P from the natural-language skill. A deterministic schema gate enforces the decidable restrictions: finite static roles, guarded recursion, finite-domain booleans, and quantifier-free linear integer arithmetic. The formal syntax and semantics are given in Appendices A–B.

Verification signals. The checker uses three static signals:

Signal	Role	Output
tool availability and effects	capability reachability	missing capability or witness action
coordination structure	global protocol conformance	protocol mismatch or conformant protocol
goal and cost refinements	logical reachability	blocked guard or model

These signals are checked before execution. Runtime payload checks remain separate obligations and are not assumed by the static checker.

Decision procedure. After the schema gate, the checker (i) rejects undeclared capabilities and goal conditions that cannot be achieved by any available capability; (ii) checks role behaviors against G, requiring exact sender choices while allowing safe receiver-side branch supersets; and (iii) explores existential protocol/world successors from W_0, using Z3 [18] for guards and refinements. A goal condition that cannot be satisfied blocks the search. If the solver times out or the pack falls outside these restrictions, the checker returns UNKNOWN rather than *impossible*. The checker therefore returns:

IMPOSSIBLE(F)	a sound explanation of why the goal cannot be reached,
ACHIEVABLE(π, O)	a witness path π and deferred obligations O,
UNKNOWN(r)	an abstention with a reason r.

Each verdict records the pack digest, checker semantics, and artifact version.

Guarantees. Let a configuration consist of a protocol state and an abstract world state, and let abs map concrete configurations to abstract configurations. Assume step and goal simulation for the declared pack (Appendix D). If no abstract run reaches φ_{goal}, no concrete run represented by that pack reaches the concrete goal (Theorem 1). The analysis remains sound when the abstraction is made less detailed, and adding capabilities cannot turn an achievable goal into an *impossible* one (Theorems 2–3). Direct conformance satisfies subject reduction and session fidelity (Theorems 4–5). The Coq development covers single-label communication with concurrent actions by other participants; world-changing interleavings require explicit stability and commutativity assumptions. The decidable restrictions above guarantee decidability. Allowing an unbounded dynamic addition of participants is an explicit undecidable extension, for which the implementation returns UNKNOWN. Full rules, assumptions, proofs, and mechanization scope appear in Appendix D.

3.1 Structured feedback and human oversight

The checker returns an explanation for IMPOSSIBLE and a witness path for ACHIEVABLE. These outputs provide concrete feedback rather than a single score. A bounded repair adapter may ask the LLM to fix a protocol, but it cannot invent capabilities or weaken the goal. Every revision is checked again by the deterministic checker. Human review remains necessary for the gap between the original intent and the formal pack. Automated prompt or scaffold search is an application of these verdicts, not an empirical contribution of this paper.

4 Implementation

The executable decision path is approximately 1.45K lines of Python, including the checker, pack gate, formula layer, and session adapter. The compaction front-end issues the artifact prompt to an LLM and passes the result through a deterministic *schema gate* before the checker sees it, so malformed packs are rejected without crashing the trusted core. The Coq artifact contains three developments and two assumption-audit harnesses: reachability soundness core (Theorems 1–3 plus `FlightInstance`, $\sim$230 lines); the direct-typing core (the head-move action safety plus `HandoffInstance`, $\sim$310 lines); and the operational-correspondence core (Theorems 4–5 for the single-label communication fragment with full interleaving, $\sim$250 lines); all three are checked in CI under pinned Coq 8.18.0 with no axioms, verified by `Print Assumptions`. The checker implements the corresponding abstractions from Appendices A–C: STRIPS frame semantics for the world, existential search over the protocol control structure, and Z3 for guard/goal satisfiability and arithmetic refinements. On success it returns the witness path; on refutation, the blocking frontier (missing capability, unsatisfiable guard, unestablished goal conjunct, or non-projectable handoff). The optional LLM front-end may use a NON_PROJECTABLE counterexample for one bounded, structure-only repair round; it may not invent capabilities or weaken the goal, and the deterministic checker remains the final referee. This is a small reflective-verification loop, not a general scaffold-search result.

The conformance step ($G \vdash (\mathbb{M}; W)$, Section C.2) is implemented by means of a simple *type inference algorithm* since all typing rules are structural in $(\mathbb{M}; W)$. At each step we freely choose either one or two participant processes to reduce or a goal satisfied by the world obtaining the set of sessions which must be typed as premises of the typing rule.

The adapter also discharges the participant side conditions of T-COMM/T-ACT/T-GOAL: it computes $\mathsf{prt}(G)$ and compares it with the roles the pack declares behaviour for. One direction refutes — a declared role outside $\mathsf{prt}(G)$ has contract End, which nothing non-trivial refines. The other cannot: a participant the pack declares nothing for offers no behaviour to refute, yet the judgment of Section C.2 ranges over a whole session with $\mathsf{prt}(G) = \mathsf{prt}(\mathbb{M})$, so the checker returns those roles alongside the verdict rather than leaving the assumption implicit.

5 Evaluation

We assembled a corpus of 15 skills spanning the canonical failure modes — hallucinated planning, retry-forever loops, coordination deadlock, refinement violations — with achievable controls (including detours and informed branches) and two deliberately *spurious* cases exercising the deferred edges. Each skill carries a ground-truth semantic label; the positive class is *achievable*. Evaluation is deterministic and CPU-only: it requires no training or live model inference, completes the 15-case matrix in under one second on a commodity laptop, and gives each individual Z3 query a 10-second timeout.

	predicted achievable	predicted impossible
truly achievable	TP $= 6$	FN $= 0$
truly impossible	FP $= 2$	TN $= 7$

Two claims, both confirmed. **Soundness**: $\mathrm{FN} = 0$ — no achievable goal was wrongly refuted, the empirical face of Theorem 1 (achievability recall $6/6 = 100\%$). **Incompleteness, contained**: $\mathrm{FP} = 2$, and both are exactly the planted spurious cases — one a false payload post-condition (Layer-C obligation), one an under-specified goal (intent edge). No structural failure was missed in this proof-of-concept corpus; this is evidence of coverage, not a proof that the two residues exhaust all empirical failure modes. Each refutation reason fires on its matching mode:

Failure mode (source)	**Verdict reason**
hallucinated planning — invokes a non-existent tool	MISSING_CAPABILITY
hallucinated planning — no establisher for a goal conjunct	GOAL_UNSAT
retry-forever loop — guard never satisfiable	BLOCKED_GUARD
coordination deadlock — unobserved choice / bad handoff	NON_PROJECTABLE
refinement violation — budget unsatisfiable on every run	GOAL_UNSAT

What the check costs. With no model in the trusted path, the checker and the deterministic front-end spend *zero* tokens; only compaction spends any, once per skill *version* and linearly in the skill's length, against a run it avoids that recurs per *invocation* and is quadratic in its turns. Under an explicit, conservative model of that run (Appendix E), compacting the 15-spec corpus costs 12.2K tokens and avoids $\sim$1.07M per round of invocations — $\sim 87\times$ leverage, unbounded deterministically — so break-even falls inside the first prevented run in every failure mode, while a skill that is *not* broken pays 2.9% of one successful run for the check, or nothing.

A separate six-case fragment suite exercises behavior absent from the headline matrix: tail-recursive retry and non-terminating control, dynamic spawning, protocol-independent refutation despite spawning, and conformant versus non-conformant declared role behavior. It yields two ACHIEVABLE, three IMPOSSIBLE, and one UNKNOWN. The abstention is reported separately rather than counted as a false negative, because UNKNOWN makes no refutation claim. Together the suites contain 21 declared packs; the 15-case matrix remains the only binary ground-truth matrix.

6 Related Work

Prior-art note. The claims below reflect the authors' survey at submission time; the fusion's novelty should be confirmed against a systematic search of the planning-meets-behavioural-types literature, which is dispersed across communities.

Multiparty session types [13, 16, 9] supply deadlock-freedom via projection and subtyping [14, 10]; our calculus and directed-choice global types follow the modern synchronous formulation of the Yoshida line [9–11]. We adopt the synthetic line [1, 2]: Section C.2 checks conformance directly against a whole session configuration rather than projected local types. Our direct rule corresponds to the conservative, plain-merge condition that bystanders behave uniformly across an unobserved choice; it does not subsume every protocol accepted by modern full merge. The new contribution is the world and goal layer: capability guards, effects, goal observation, and the asymmetric refutation guarantee are not modelled by classical MPST or new SMPS. *Automated planning* [17] supplies capability-guarded goal reachability; we reuse its frame semantics but embed it in a multiparty choreography. The contribution is the *fusion*, decided over an goal-marked global type synthesized by an LLM. Recent agent-verification work is adjacent but differently aimed: *provable coordination via message sequence charts* [20] projects a global choreography for LLM agents but targets coordination correctness, not goal achievability; its runtime-planning extension also establishes that LLM synthesis of a coordination protocol is itself prior art. *VeriGuard* [21] separates offline policy verification from online monitoring as a safety shield; *agent behavioural contracts* [22] provide probabilistic, runtime-enforced compliance and composition conditions for multi-agent chains, whereas our verdict is static and sound for refutation relative to the pack.

Skill-bundle scanners and verified-skill catalogs are complementary. NVIDIA's SkillSpector [23] and Verified Agent Skills pipeline [24] address the same deployment object—portable agent skills—but ask a different question. SkillSpector is a pre-publication security control: it normalizes a skill bundle and runs deterministic scanners such as static pattern detectors, AST and taint analyzers, manifest-consistency checks, metadata-poisoning checks, supply-chain checks, and optional LLM-backed semantic analyzers. This is valuable admission control for skills, but it is not a compiler or type discipline in the sense used here. As published, SkillSpector does not aim to provide an operational semantics of goal-marked skills, a goal-reachability decision, or a refutation-soundness theorem. A SkillSpector-like pass belongs at the boundary of the compiler: before or alongside LLM compaction, it can vet the incoming `SKILL.md`, code, dependencies, and MCP metadata; after compaction, it can help justify the honesty of declared capabilities and least-privilege metadata. The type checker then consumes the sanitized formal pack and proves the narrower claim that the declared goal is not structurally impossible.

None of the above frames the static check as goal achievability over a declared world and goal-marked protocol, nor gives the same refutation-sound/achievement-incomplete asymmetry.

7 Limitations and Future Work

- **Relative soundness.** Everything is proved about the declared Γ and S. If the prose lies (a "read-only" tool that writes), the checker verifies a fiction; honest-declaration is a separate obligation, of which this makes only the interaction slice decidable. A practical compiler should therefore include a skill-bundle security pass—for example a SkillSpector-like scanner—to inspect `SKILL.md`, scripts, dependencies, manifests, and permission metadata before the formal pack is trusted by the achievability checker.
- **Static topology for totality.** Dynamic spawning drops the procedure to a semi-decision (Proposition 2); a practical system answers UNKNOWN rather than guessing.
- **Trusted abstraction.** A coarse α silently widens tolerance (more false achievable), never breaking Theorem 1 but weakening usefulness.
- **Engineering gaps.** The mechanization covers the generic refutation-sound abstraction theorem and concrete examples, head-move action safety, and subject reduction/session fidelity with full bystander interleaving for the single-label communication model, all axiom-free. The executable checker is schema-gated and tested against that specification, but the STEP-SIM/GOAL-SIM instantiation for its exact symbolic state is not mechanized. The Coq correspondence model also does not yet combine observable goal labels, participant matching, multi-branch choice, and world-changing interleaving in one development. Still open are that faithful mechanization; a head-normalization/permutation bridge from interleaved session runs to the head-ordered reachability search; equivalence between the declarative judgment and the projection-based adapter; and a concrete-session realization theorem for abstract witnesses. The 21-pack evaluation remains a proof of concept.

Broader impact. Pre-execution refutation reduces wasted computation (Appendix E) and prevents structurally impossible agent plans from reaching deployment. The same formal verdict can, however, create false confidence when an inaccurate pack omits a harmful tool effect or weakens the user's actual goal. We therefore expose the declared pack, witness or blocking frontier, and deferred obligations rather than presenting a single trust score. Intent review and runtime payload monitoring remain necessary before the method is used in consequential settings; this work does not evaluate fairness, privacy, or deployment safety.

8 Conclusion

An LLM can draft an agent's plan from intent; the open question has been whether anything can tell, cheaply and before execution, that the plan is doomed. For the *achievability* question the answer is yes, relative to a declared pack: combine planning's capability-guarded reachability with a synthetic, directly-typed global protocol; treat LLM compaction as untrusted; and use a refutation-sound abstraction so an *impossible* verdict is never wrong about that declared model. *Achievable* remains honest about payload and intent fidelity, while *Unknown* abstains outside static topology. This yields a structured, inspectable verification signal for agent skills and a path toward, rather than a completed proof of, stronger per-step safety.

References

[1] Franco Barbanera, Mariangiola Dezani-Ciancaglini & Ugo de'Liguoro (2024): *Un-projectable Global Types for Multiparty Sessions*. In Alessandro Bruni, Alberto Momigliano, Matteo Pradella & Matteo Rossi, editors: *PPDP*, ACM Press, pp. 15:1–15:13,

[2] Francesco Dagnino, Paola Giannini & Mariangiola Dezani-Ciancaglini (2023): *Deconfined Global Types for Asynchronous Sessions*. Logical Methods in Computer Science 19(1), pp. 1–41,

[3] J. Li, X. Xiao, Y. Zhang, C. Liu, L. Zhao, X. Liao, Y. Ji, J. Wang, J. Gu, Y. Ge, W. Xu, X. Fang, X. Xu, T. Zhao, Y. Kim, T. Wang, J. Hamm, S. Krishnaswamy, J. Huan, C. Reddy. Agent Harness Engineering: A Survey. 2026.

[4] M. Chen, J. Wang, Z. Liu, Y. Wang, H. Zheng, Q. Wang. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. arXiv:2606.06324, 2026.

[5] Agent Skills Standard. Specification. `https://github.com/agentskills/agentskills/blob/main/docs/specification.mdx`, 2025.

[6] S. Hu, C. Lu, J. Clune. Automated Design of Agentic Systems. *ICLR*, 2025.

[7] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, et al. AFlow: Automating Agentic Workflow Generation. *ICLR*, 2025.

[8] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, I. Stoica. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025.

[9] N. Yoshida, L. Gheri. A Very Gentle Introduction to Multiparty Session Types. *ICDCIT*, LNCS 11969, 2020.

[10] S. Ghilezan, S. Jakšić, J. Pantović, A. Scalas, N. Yoshida. Precise Subtyping for Synchronous Multiparty Sessions. *J. Log. Algebr. Meth. Program.* 104, 2019.

[11] A. Scalas, N. Yoshida. Less is More: Multiparty Session Types Revisited. *Proc. ACM Program. Lang.* 3(POPL), 2019.

[12] K. Honda, V. T. Vasconcelos, M. Kubo. Language Primitives and Type Discipline for Structured Communication-Based Programming. *ESOP*, LNCS 1381, pp. 122–138, 1998.

[13] K. Honda, N. Yoshida, M. Carbone. Multiparty Asynchronous Session Types. *J. ACM* 63(1), 2016.

[14] S. Gay, M. Hole. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42(2–3), 2005.

[15] D. Brand, P. Zafiropulo. On Communicating Finite-State Machines. *J. ACM* 30(2), 1983.

[16] E. Li, F. Stutz, T. Wies, D. Zufferey. Complete Multiparty Session Type Projection with Automata. *CAV* 2023.

[17] R. Fikes, N. Nilsson. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artif. Intell.* 2(3–4), 1971.

[18] L. de Moura, N. Bjørner. Z3: An Efficient SMT Solver. *TACAS*, LNCS 4963, pp. 337–340, Springer, 2008.

[19] C. Barrett, P. Fontaine, C. Tinelli. The SMT-LIB Standard: Version 2.6. Technical Report, Department of Computer Science, The University of Iowa, 2017. `https://smt-lib.org`.

[20] B. Bollig, M. Függer, T. Nowak. Provable Coordination for LLM Agents via Message Sequence Charts. *ISoLA*, 2026. arXiv:2604.17612.

[21] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, L. T. Le. VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. arXiv:2510.05156, 2025.

[22] V. P. Bhardwaj. Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents. arXiv:2602.22302, 2026.

[23] N. Paz, K. Pradeep, N. Raghavan, A. Nikirk, M. Gupta. SkillSpector: A Pre-Publication Security Control for Agent Skills. *Agent Skills Workshop at CAIS*, 2026.

[24] NVIDIA. NVIDIA-Verified Agent Skills Provide Capability Governance for AI Agents. NVIDIA Technical Blog, 19 May 2026.

A Syntax

We fix disjoint sets of predicates $p \in \mathsf{Pred}$ (boolean), numeric variables $x \in \mathsf{Var}$, roles $\mathsf{p}, \mathsf{q}, \mathsf{r} \in \mathcal{R}$, capability names $a \in \mathsf{Cap}$, and message labels $\ell \in \mathsf{Lab}$. Following the modern presentation of multiparty sessions [1, 2] we proceed bottom-up: state and capabilities (Sections A.1–A.2), then *processes* — the programs skills execute as — (Section A.3), then the global *type* they are checked against directly (Section A.4). In each grammar, "::=" lists the possible forms of an expression. In each inference rule below, the statement beneath the line follows when every premise above the line holds. Rule names identify the corresponding checker obligation; they are not additional assumptions.

A.1 State logic

$$
\begin{array}{rcll}
e & ::= & x \mid n \mid e+e \mid e-e \mid c \cdot e & (c, n \in \mathbb{Z}) \\
\varphi & ::= & p \mid \top \mid \bot \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid e \bowtie e & \bowtie \in \{<, \leq, =, \geq, >, \neq\}
\end{array}
$$

The state logic $\mathcal{L}$ is quantifier-free linear integer arithmetic [19] with uninterpreted boolean predicates — a decidable fragment for which entailment $\vDash$ and satisfiability are discharged by a satisfiability-modulo-theories (SMT) solver.

A.2 Worlds and capabilities

A *world* $W = \langle B, N \rangle$ assigns truth values to predicates $B : \mathsf{Pred} \to \mathbb{B}$ and numeric variables $N : \mathsf{Var} \to \mathbb{Z}$. A *capability context* Γ maps each available tool a to a guarded effect:

$$
\begin{array}{c}
\Gamma(a) = \langle pre_a, \mathit{eff}_a \rangle, \qquad pre_a \in \mathcal{L}, \\
\mathit{eff}_a = \langle Add_a \subseteq \mathsf{Pred},\ Del_a \subseteq \mathsf{Pred},\ Asg_a : \mathsf{Var} \rightharpoonup e,\ Nd_a : \mathsf{Var} \rightharpoonup \mathcal{L} \rangle \\
a \notin \mathrm{dom}(\Gamma) \quad \iff \quad \text{the agent does } \textit{not} \text{ possess tool } a \quad \text{(no hallucinated tools)}.
\end{array}
$$

Add/Del are the STRIPS add- and delete-lists; Asg gives deterministic numeric updates $x := e$; Nd gives *nondeterministic* updates $x := *$ constrained by a formula over the new value (used to model post-conditions such as *"books a fare under 500"*).

A.3 Processes and multiparty sessions

Skills *execute* as processes; a multi-agent run is a session of named processes. We use the synchronous multiparty session calculus in its modern coinductive presentation [1, 2]: processes coinductively defined are possibly infinite *regular* trees (finitely many distinct subtrees), and labels are goal-relevant tokens, $\ell = \alpha(m)$ for concrete messages m — the tolerance dial α is a parameter of the label alphabet.

$$
P \ ::=^{coind}\ \mathbf{0} \ \mid\ \mathsf{p}!\{\ell_i.\, P_i\}_{i \in I} \ \mid\ \mathsf{p}?\{\ell_i.\, P_i\}_{i \in I} \ \mid\ a.\, P \qquad\qquad \mathbb{M} \ =\ \mathsf{p}_1[P_1] \parallel \cdots \parallel \mathsf{p}_n[P_n]
$$

with I finite and non-empty, labels ℓ_i pairwise distinct, and role names pairwise distinct. The output $\mathsf{p}!\{\ell_i.P_i\}$ *internally* chooses a label and sends it to p; the input $\mathsf{p}?\{\ell_i.P_i\}$ offers an *external* choice; $a.P$ invokes capability a — the only construct that changes the world. A skill S_{p} *is* a process: the behaviour declared for role p.

A.4 Global types

The single communication construct of a global type is a *directed* labelled choice — selection by p and branching at q in one construct. (A partnerless "p selects" admits no coherent view for the roles that did not choose; directed choice is the standard MPST construct [13, 9].)

$$
G \ ::=^{coind}\ \mathsf{p} \to \mathsf{q} : \{\ell_i.\, G_i\}_{i \in I} \ \mid\ a@\mathsf{p}.\, G \ \mid\ \checkmark\varphi.\, G \ \mid\ \mathsf{End}
$$

subject to $\mathsf{p} \neq \mathsf{q}$ and the same regularity and label conditions as processes. This is the *only* type in the discipline: Section C.2 types a whole session directly against G — there is no local-type grammar, no projection function, and no separate subtyping relation. Goal markers $\checkmark\varphi$ live *only* in G: achievability is decided on the global type (Rule ACH in C.3), and markers are consumed at typing time by T-GOAL without ever burdening a process. Only $a@\mathsf{p}$ changes the world; messages carry coordination information only.

B Operational Semantics

B.1 World transitions

A capability fires when its guard holds, producing a successor world. Because Nd is nondeterministic, $\langle W\rangle\, a\, \langle W'\rangle$ may relate one world to many. The relation is implicitly indexed by Γ: the notation pre_a, eff_a is defined only when $\Gamma(a) = \langle pre_a, \mathit{eff}_a\rangle$, so every firing presupposes $a \in \mathrm{dom}(\Gamma)$.

$$\textsc{World-Act}\quad \frac{W \vDash pre_a \qquad W' \in \mathrm{apply}(\mathit{eff}_a, W)}{\langle W\rangle\ a\ \langle W'\rangle}$$

$$\mathrm{apply}(\mathit{eff}_a, \langle B, N\rangle) = \langle B', N'\rangle, \qquad B' = (B \cup Add_a) \setminus Del_a$$

$$N'(x) = \begin{cases} [\![Asg_a(x)]\!]_N & \text{if } x \in \mathrm{dom}(Asg_a) \\ v \text{ with } \langle B, N[x \mapsto v]\rangle \vDash Nd_a(x) & \text{if } x \in \mathrm{dom}(Nd_a) \\ N(x) & \text{otherwise (frame)} \end{cases}$$

B.2 A labelled semantics for sessions and global types

Only capabilities touch the world, so sessions reduce as configurations $(\mathbb{M}; W)$; communication is synchronous [10]. Each transition carries a label Λ recording its *participants*, so that independent moves by disjoint roles can be identified. Two auxiliary maps drive the labelled system: the *participants* $\mathsf{prt}(\cdot)$ of a global type, a label, or a session, and the *capabilities* $\mathsf{cap}(\cdot)$ (the set of moves a global type can perform). On global types,

$$\mathsf{prt}(\mathsf{p}{\to}\mathsf{q} : \{\ell_i.\, G_i\}_{i\in I}) = \{\mathsf{p}, \mathsf{q}\} \cup \textstyle\bigcup_{i\in I} \mathsf{prt}(G_i) \qquad \mathsf{prt}(\checkmark\varphi.G) = \mathsf{prt}(G)$$
$$\mathsf{prt}(a@\mathsf{p}.G) = \{\mathsf{p}\} \cup \mathsf{prt}(G) \qquad \mathsf{prt}(\mathsf{End}) = \emptyset,$$

on labels $\mathsf{prt}(\mathsf{p}\,\ell\,\mathsf{q}) = \{\mathsf{p}, \mathsf{q}\}$, $\mathsf{prt}(a@\mathsf{p}) = \{\mathsf{p}\}$, $\mathsf{prt}(\checkmark\varphi) = \emptyset$, and on sessions $\mathsf{prt}(\mathbb{M}) = \{\mathsf{p} \mid \mathbb{M} \equiv \mathsf{p}[P]\ \&\ P \neq \mathbf{0}\}$ (the roles with a residual behaviour). The capabilities of a global type are

$$\mathsf{cap}(\mathsf{p}{\to}\mathsf{q} : \{\ell_i.\, G_i\}_{i\in I}) = \{\mathsf{p}\,\ell_i\,\mathsf{q} \mid i \in I\} \cup \textstyle\bigcup_{i\in I} \mathsf{cap}(G_i)$$
$$\mathsf{cap}(a@\mathsf{p}.G) = \{a@\mathsf{p}\} \cup \mathsf{cap}(G) \qquad \mathsf{cap}(\checkmark\varphi.G) = \{\checkmark\varphi\} \cup \mathsf{cap}(G) \qquad \mathsf{cap}(\mathsf{End}) = \emptyset.$$

The capability names used by a protocol are

$$\mathsf{caps}(G) = \{a \mid \exists \mathsf{p}.\ a@\mathsf{p} \in \mathsf{cap}(G)\}.$$

The LTS of sessions is defined by

$$\textsc{S-Comm}\quad \frac{k \in I \subseteq J}{(\mathsf{p}[\mathsf{q}!\{\ell_i.P_i\}_{i\in I}] \parallel \mathsf{q}[\mathsf{p}?\{\ell_j.Q_j\}_{j\in J}] \parallel \mathbb{M}; W) \xrightarrow{\mathsf{p}\,\ell_k\,\mathsf{q}} (\mathsf{p}[P_k] \parallel \mathsf{q}[Q_k] \parallel \mathbb{M}; W)}$$

$$\textsc{S-Act}\quad \frac{\langle W\rangle\ a\ \langle W'\rangle}{(\mathsf{p}[a.P] \parallel \mathbb{M}; W) \xrightarrow{a@\mathsf{p}} (\mathsf{p}[P] \parallel \mathbb{M}; W')} \qquad \textsc{S-Goal}\quad \frac{W \vDash \varphi}{(\mathbb{M}; W) \xrightarrow{\checkmark\varphi} (\mathbb{M}; W)}$$

where $\parallel$ is commutative, associative and $\mathsf{p}[\mathbf{0}] \parallel \mathbb{M} \equiv \mathbb{M}$. A non-terminated configuration with no successor is *stuck*; the deadlocking planner/worker handoff of Section 1 is the stuck two-role configuration in which both processes begin with an input. A goal marker is *not* carried by any process; instead S-Goal lets a configuration *observe* that its world already entails one of its declared goals φ, emitting the label $\checkmark\varphi$ without altering $\mathbb{M}$ or W. (Each session carries a set of goal formulas, and φ ranges over that set — not over every formula the world happens to satisfy — so that each observation has a matching marker in the protocol.) This observable goal step is matched at the type level by G-Goal-E below, so that $\checkmark\varphi$ is a genuine label Λ of both transition systems and the operational correspondence of Section D.3 ranges over it uniformly.

The checker relates a session to its protocol through a *labelled* transition system on global types, $(G, W) \rightsquigarrow_\Lambda (G', W')$, built from *head* rules that consume the leading construct and *interleaving* rules that let a move by participants *disjoint* from the head commute inward:

$$\textsc{G-Comm-E}\ \frac{k \in I}{(\mathsf{p}{\to}\mathsf{q}{:}\{\ell_i.G_i\}_{i\in I},\, W) \rightsquigarrow_{\mathsf{p}\,\ell_k\,\mathsf{q}} (G_k,\, W)} \qquad \textsc{G-Act-E}\ \frac{\langle W\rangle\, a\, \langle W'\rangle}{(a@\mathsf{p}.\, G,\, W) \rightsquigarrow_{a@\mathsf{p}} (G,\, W')}$$

$$\textsc{G-Goal-E}\ \frac{W \vDash \varphi}{(\checkmark\varphi.\, G,\, W) \rightsquigarrow_{\checkmark\varphi} (G,\, W)}$$

$$\textsc{G-Comm-I}\ \frac{(G_i, W) \rightsquigarrow_\Lambda (G_i', W') \qquad \Lambda \in \mathsf{cap}(G_i)\ \ \forall i \in I \qquad \mathsf{prt}(\Lambda) \cap \{\mathsf{p}, \mathsf{q}\} = \emptyset}{(\mathsf{p}{\to}\mathsf{q}{:}\{\ell_i.G_i\}_{i\in I},\, W) \rightsquigarrow_\Lambda (\mathsf{p}{\to}\mathsf{q}{:}\{\ell_i.G_i'\}_{i\in I},\, W')}$$

$$\textsc{G-Act-I}\ \frac{(G, W) \rightsquigarrow_\Lambda (G', W') \qquad \Lambda \in \mathsf{cap}(G) \qquad \mathsf{prt}(\Lambda) \cap \{\mathsf{p}\} = \emptyset}{(a@\mathsf{p}.\, G,\, W) \rightsquigarrow_\Lambda (a@\mathsf{p}.\, G',\, W')}$$

$$\textsc{G-Goal-I}\ \frac{(G, W) \rightsquigarrow_\Lambda (G', W') \qquad \Lambda \in \mathsf{cap}(G)}{(\checkmark\varphi.\, G,\, W) \rightsquigarrow_\Lambda (\checkmark\varphi.\, G',\, W')}$$

G-Goal-E discharges a goal marker when $W \vDash \varphi$, emitting the observable label $\checkmark\varphi$ that matches S-Goal. In the head-only reduction used for achievability, an unsatisfied marker is a checkpoint and blocks the path. In the full labelled correspondence, G-Goal-I lets a continuation step commute inward under a still-pending marker. When that step changes the world, the correspondence theorem uses the goal-stability hypothesis stated in Section D.3; removing that hypothesis is future work. The extraction (head) rules G-Comm-E/G-Act-E/G-Goal-E give the Λ-erased reduction $(G, W) \rightsquigarrow (G', W')$ that the achievability judgment below uses; the interleaving rules matter only for the operational correspondence of Section D.3, and their $\Lambda \in \mathsf{cap}(G)$ premise confines a commuting move to a capability the residual protocol actually offers. G-Comm-E is legitimate because choice is directed — the branch is a communication both endpoints observe, and Section C.2 checks that every third party can follow. A well-formed pack uses its declared goal φ_{goal} at each goal marker. A configuration is *goal-satisfying* when control is at such a marker or at the terminus and the world entails that declared goal:

$$goal_{\varphi_{goal}}(\checkmark\varphi_{goal}.G, W) \iff W \vDash \varphi_{goal}, \qquad goal_{\varphi_{goal}}(\mathsf{End}, W) \iff W \vDash \varphi_{goal},$$
$$\Gamma; G \vDash \Diamond_{init}\varphi_{goal} \iff \exists W_0 \vDash init.\ \exists (G', W').\ (G, W_0) \rightsquigarrow^* (G', W') \ \wedge\ goal_{\varphi_{goal}}(G', W').$$

The diamond is *existential*: a single witnessing run suffices — the formal source of detour-tolerance.

B.3 Realizability and subtyping, without projection

The classical route to ruling out a deadlocking handoff is *projection*: compute each role's local type $G{\restriction}\mathsf{r}$ by structural recursion on G, taking the *merge* of a choice's branches at every role that neither sends nor receives, and declaring G *realizable* exactly when this partial function is defined everywhere — undefinedness of the merge is the static signature of a role forced to behave differently in branches it cannot distinguish. Session subtyping $\leq$ is then a second, separate coinductive relation on local types, letting a role's actual behaviour offer more external choices and fewer internal ones than its projected contract demands [14, 10, 11].

Both devices exist to answer questions about the *network as a whole* — "can every role tell the branches apart it needs to?", "may this implementation stand in for that contract?" — while type-checking only one role's syntax at a time. Section C.2 answers the same two questions with a single judgment, *typing the whole configuration directly against* G: a choice node's rule quantifies over every branch the protocol declares and requires the *same* residual configuration to check against each one. This is the conservative plain-merge condition, obtained without ever computing $\sqcap$ or a local type. The receiver-side subtyping direction (a receiver may accept a superset of the protocol's labels) is folded into the same rule as a label-set inclusion $I \subseteq J$. Tolerance therefore still has the same three independent knobs from Section 2 — the existential $\Diamond$ (Section B.2), subtyping (now inside

T-Comm, Section C.2), and the granularity of α — "flexible, tolerant of small details" is still the profile $\langle$coarse α, $\Diamond$, linear $\mathcal{L}\rangle$; a later safety layer flips the same machinery to the universal $\Box$ over permission predicates.

C The Achievability Type System

Achievability now factors into three premises, each separately decidable, combined by the top rule Ach: no hallucinated capability, direct conformance of the declared skills to G, and reachability of the goal.

C.1 Capability and goal typing

$$\textsc{T-Eff}\ \frac{\Gamma(a) = \langle \mathit{pre}, \mathit{eff}\rangle \qquad \varphi \vDash \mathit{pre}}{\Gamma \vdash \{\varphi\}\, a\, \{\mathrm{sp}(\varphi, \mathit{eff})\}} \qquad\qquad \textsc{T-Miss}\ \frac{a \notin \mathrm{dom}(\Gamma)}{\Gamma \vdash a \,\triangleright\, \text{✗}\, (\textsc{Missing\_Capability})}$$

$\mathrm{sp}(\varphi, \mathit{eff})$ is the strongest postcondition of the STRIPS effect. T-Miss fires on hallucinated planning: the plan names a tool the context does not grant, and achievability is refuted without any search.

C.2 Direct conformance typing

Skills are typed *directly* against the global protocol and the world, by the coinductive judgment $G \vdash (\mathbb{M}; W)$ — read "G types session $\mathbb{M}$ starting from world W." There is no local type, no projection, and no separate subtyping relation: the receiver-side subtyping direction and the unobserved-choice check are structural side conditions of one rule.

$$\textsc{T-End}\ \frac{}{\mathsf{End} \vdash (\mathsf{p}[\mathbf{0}]; W)} \qquad \textsc{T-Act}\ \frac{G \vdash (\mathsf{p}[P] \parallel \mathbb{M}; W') \qquad \langle W\rangle\, a\, \langle W'\rangle \qquad \mathsf{prt}(G) \cup \{\mathsf{p}\} = \mathsf{prt}(\mathbb{M}) \cup \{\mathsf{p}\}}{a@\mathsf{p}.\, G \vdash (\mathsf{p}[a.P] \parallel \mathbb{M}; W)}$$

$$\textsc{T-Goal}\ \frac{G \vdash (\mathbb{M}; W) \qquad W \vDash \varphi \qquad \mathsf{prt}(G) = \mathsf{prt}(\mathbb{M})}{\checkmark\varphi.\, G \vdash (\mathbb{M}; W)}$$

$$\textsc{T-Comm}\ \frac{G_i \vdash (\mathsf{p}[P_i] \parallel \mathsf{q}[Q_i] \parallel \mathbb{M}; W) \qquad \forall i \in I \subseteq J \qquad \mathsf{prt}(\mathsf{p}{\to}\mathsf{q} : \{\ell_i.G_i\}_{i\in I}) = \mathsf{prt}(\mathbb{M}) \cup \{\mathsf{p},\mathsf{q}\}}{\mathsf{p}{\to}\mathsf{q} : \{\ell_i.G_i\}_{i\in I} \vdash (\mathsf{p}[\mathsf{q}!\{\ell_i.P_i\}_{i\in I}] \parallel \mathsf{q}[\mathsf{p}?\{\ell_j.Q_j\}_{j\in J}] \parallel \mathbb{M}; W)}$$

T-Act pairs type and world in lockstep with World-Act: the unconsumed type $a@\mathsf{p}.G$ sits with the *pre*-effect world W and the residual G with the *post*-effect world W' — the same convention as G-Act-E (Section B.2). In T-Comm the sender p offers exactly the protocol's branch set I, while the receiver q may accept *more* ($I \subseteq J$) — the one subtyping direction the rule keeps (safe interface slack at the receiver). Requiring *every* protocol branch ℓ_i to be checked against the *same* bystanders $\mathbb{M}$ is the plain-merge form of the unobserved-choice condition, so a role forced to behave differently across branches it cannot distinguish makes no derivation possible — the rule fails closed, and deadlock-freedom falls out of T-Comm itself with no merge to compute (Theorem 4). Each rule additionally carries a *participant-matching* side condition tying the protocol's participants to the session's: $\mathsf{prt}(\mathsf{p}{\to}\mathsf{q} : \{\ell_i.G_i\}) = \mathsf{prt}(\mathbb{M}) \cup \{\mathsf{p},\mathsf{q}\}$ in T-Comm, $\mathsf{prt}(G) \cup \{\mathsf{p}\} = \mathsf{prt}(\mathbb{M}) \cup \{\mathsf{p}\}$ in T-Act, and $\mathsf{prt}(G) = \mathsf{prt}(\mathbb{M})$ in T-Goal. These make the invariant "the roles the protocol still mentions are exactly the roles still running" hold at every node, ruling out a session that carries a stray participant the protocol has forgotten (or vice versa) — the well-formedness that lets the interleaving cases of Section D.3 match a bystander move on the session to a $\mathsf{cap}(\cdot)$-labelled move on the type. T-End closes the discipline at the terminus: the terminated protocol End types a single idle role $\mathsf{p}[\mathbf{0}]$ — and, through the congruence $\mathsf{p}[\mathbf{0}] \parallel \mathbb{M} \equiv \mathbb{M}$, any session all of whose roles have terminated — the base case of the participant invariant, since $\mathsf{prt}(\mathsf{End}) = \emptyset$ and an idle role contributes nothing to $\mathsf{prt}(\mathbb{M})$. T-Act is derivable only when $\langle W\rangle\, a\, \langle W'\rangle$ holds, which itself presupposes $a \in \mathrm{dom}(\Gamma)$: this is where hallucinated capabilities die, with T-Miss as the diagnostic.

C.3 The achievability judgment

$$\frac{\textsc{Ach}\quad \mathsf{caps}(G) \subseteq \mathrm{dom}(\Gamma) \qquad \exists W_0 \vDash \mathit{init}.\ G \vdash (\mathsf{p}_1[S_{\mathsf{p}_1}] \parallel \cdots \parallel \mathsf{p}_n[S_{\mathsf{p}_n}]; W_0) \ \wedge\ \Gamma; (G, W_0) \vDash \Diamond\varphi_{goal}}{\Gamma \vdash \{S_{\mathsf{p}}\}_{\mathsf{p}} \ :\ G \rhd \Diamond\varphi_{goal}}$$

Read top-to-bottom: no hallucinated tools; some plausible initial world exists from which the declared skills directly conform to the protocol (deadlock-free, every capability call guarded) *and* that same world reaches the goal. The reachability half is unchanged from Section B.2 and decided on Γ, G alone, before any S_{p} is known — exactly what lets the checker run on the LLM-compacted pack the moment it exists; conformance is the additional, separately decidable check once the skills are declared, and needs no transport lemma: the receiver-side subtyping slack is already inside T-COMM.

C.4 Reachability rules

The diamond is derived by the following inductive system, realized by the checker as a finite forward search over $\rightsquigarrow$; unlike T-COMM/T-ACT/ T-GOAL, it never mentions a session $\mathbb{M}$, which is what makes it decidable on the pack alone.

$$\frac{\textsc{Reach-Goal}\quad W \vDash \varphi_{goal} \qquad G = \mathsf{End} \ \vee\ \exists G'.\, G = \checkmark\varphi_{goal}.G'}{\Gamma; (G, W) \vDash \Diamond\varphi_{goal}}$$

$$\frac{\textsc{Reach-Step}\quad (G, W) \rightsquigarrow (G', W') \qquad \Gamma; (G', W') \vDash \Diamond\varphi_{goal}}{\Gamma; (G, W) \vDash \Diamond\varphi_{goal}} \qquad \frac{\textsc{Reach-Or}\quad \exists k \in I.\, \Gamma; (G_k, W) \vDash \Diamond\varphi_{goal}}{\Gamma; (\mathsf{p}{\to}\mathsf{q}{:}\{\ell_i.G_i\}_{i\in I}, W) \vDash \Diamond\varphi_{goal}}$$

D Metatheory

We separate what is *mechanized in Coq* (Theorems 1–3; Theorems 4–5 for the single-label communication fragment with full interleaving; the head-move action case; and the concrete instances, all axiom-free) from what is *proved on paper* (the action-interleaving cases, decidability, and the undecidability boundary).

D.1 Soundness of the abstraction

Let $\mathcal{C}$ be the concrete configuration space of protocol/world pairs, $\mathcal{A}$ the abstract configuration space the checker explores, and $abs : \mathcal{C} \to \mathcal{A}$ the abstraction. We require:

$$\textbf{(step-sim)}\ \ C \rightsquigarrow C' \Rightarrow abs(C) \mathrel{\hat{\rightsquigarrow}} abs(C'), \qquad \textbf{(goal-sim)}\ \ goal_{\varphi_{goal}}(C) \Rightarrow \hat{goal}(abs(C)).$$

Lemma 1 (Transport)**.** *If $C_0 \rightsquigarrow^* C$ then $abs(C_0) \mathrel{\hat{\rightsquigarrow}^*} abs(C)$.*

Proof. By induction on the reflexive-transitive closure. The empty run is reflexivity. For $C_0 \rightsquigarrow^* C' \rightsquigarrow C$, the induction hypothesis gives $abs(C_0) \mathrel{\hat{\rightsquigarrow}^*} abs(C')$ and STEP-SIM gives $abs(C') \mathrel{\hat{\rightsquigarrow}} abs(C)$; compose them. Mechanized as `reach_abs`, parameterized by STEP-SIM. □

Lemma 2 (Reachability monotonicity)**.** *If every step of $\rightsquigarrow_1$ is a step of $\rightsquigarrow_2$ (that is, $x \rightsquigarrow_1 y$ implies $x \rightsquigarrow_2 y$), then $s \rightsquigarrow_1^* w$ implies $s \rightsquigarrow_2^* w$.*

Proof. By induction on the closure $s \rightsquigarrow_1^* w$. The empty run transports by reflexivity. For $s \rightsquigarrow_1^* u \rightsquigarrow_1 w$, the induction hypothesis gives $s \rightsquigarrow_2^* u$; the hypothesis turns the last step $u \rightsquigarrow_1 w$ into $u \rightsquigarrow_2 w$; appending it gives $s \rightsquigarrow_2^* w$. Mechanized as `reach_mono`. □

D.2 Refutation soundness, tolerance, monotonicity

Theorem 1 (Refutation soundness)**.** *If the abstract system has no reachable goal state from $abs(G, W_0)$, then no declared run of that pack configuration reaches φ_{goal}. Hence an* impossible *verdict is never wrong relative to the pack, provided* STEP-SIM *and* GOAL-SIM *hold.*

Proof. We prove the contrapositive: if *some* declared run reaches the goal, then the abstract system reaches an abstract goal state. Suppose $C_0 \rightsquigarrow^* C$ with $goal_{\varphi_{goal}}(C)$. By Lemma 1, $abs(C_0) \hat{\rightsquigarrow}^* abs(C)$, and by GOAL-SIM, $\widehat{goal}(abs(C))$. Thus $abs(C)$ is an abstract goal state reachable from $abs(C_0)$, contradicting the hypothesis. Hence no declared run reaches φ_{goal}: a verdict of *impossible* — read off exactly the premise, that the abstract search exhausts without hitting an abstract goal — is never wrong. Mechanized, axiom-free as a theorem schema parameterized by the two simulation hypotheses (the existential witness is $abs(C)$): ☐

```
Theorem refutation_sound :  forall s0,
(~ exists a, reach astep (abs s0) a /\ agoal a) ->
(~ exists w, reach cstep s0 w /\ cgoal w).
(* Print Assumptions refutation_sound.  => Closed under the global
context *)
```

Theorem 2 (Tolerance soundness)**.** *If a* coarser *over-approximation (more edges) cannot reach the goal, neither can any finer system. Hence coarsening α to tolerate more detail never produces a false refutation.*

Proof. Let $\rightsquigarrow_{\text{fine}}$ and $\rightsquigarrow_{\text{coarse}}$ be the two abstract step relations, with the coarsening an *over-approximation*: every fine step is also a coarse step, $x \rightsquigarrow_{\text{fine}} y \Rightarrow x \rightsquigarrow_{\text{coarse}} y$. Suppose, for contradiction, that the finer system reaches the goal: $s_0 \rightsquigarrow^*_{\text{fine}} a$ with $\widehat{goal}(a)$. By Lemma 2 (with $\rightsquigarrow_1 = \rightsquigarrow_{\text{fine}}$, $\rightsquigarrow_2 = \rightsquigarrow_{\text{coarse}}$), $s_0 \rightsquigarrow^*_{\text{coarse}} a$; and $\widehat{goal}(a)$ still holds. So the coarser system reaches a goal state, contradicting the hypothesis. Hence no coarser-refuted goal is reachable in any finer system: coarsening α only *adds* abstract edges, so it can never turn an unreachable goal into a reachable one — refutation is preserved. Mechanized as `tolerance_sound`, axiom-free. ☐

Theorem 3 (Capability monotonicity)**.** *If $\Gamma \subseteq \Gamma'$ and Γ reaches φ_{goal}, then Γ' reaches φ_{goal}. Granting tools never makes an achievable goal impossible.*

Proof. Enlarging the capability context only *adds* guarded effects: since $\Gamma \subseteq \Gamma'$, every capability firing available under Γ is available under Γ' with the same guard and effect (WORLD-ACT in B.1 quantifies over $a \in \mathrm{dom}(\Gamma) \subseteq \mathrm{dom}(\Gamma')$), so every step of the Γ-transition relation is a step of the Γ'-relation. Suppose Γ reaches the goal configuration C from C_0. By Lemma 2, the same configuration is reachable under Γ', and $goal_{\varphi_{goal}}(C)$ is unchanged because the goal predicate does not depend on Γ. Hence Γ' reaches φ_{goal} via the same witnessing run. The other premises of ACH are monotone in the same direction ($\mathsf{caps}(G) \subseteq \mathrm{dom}(\Gamma) \subseteq \mathrm{dom}(\Gamma')$ still holds, and realizability and conformance do not mention Γ beyond $a \in \mathrm{dom}(\Gamma)$), so the whole judgment is preserved. The transport half is mechanized as `cap_monotone`, axiom-free, stated over an inclusion of step relations; the reduction of $\Gamma \subseteq \Gamma'$ to that inclusion is the WORLD-ACT argument above, on paper. ☐

Concrete instance. The flight skill of Section 2, variant A, is mechanized as `FlightInstance`. We prove `flight_refuted` (no run reaches booked $\wedge$ confirmation_sent, since no step touches conf) and `booking_reachable` (a booking is still reachable). Both are axiom-free, witnessing non-vacuity of Theorem 1.

D.3 Operational correspondence: subject reduction and session fidelity

The direct judgment $G \vdash (\mathbb{M}; W)$ is not merely preserved by reduction: the session and its protocol step *in lockstep* over the labelled transition systems of Section B.2. This is the standard soundness backbone of a session calculus [13, 11], obtained here *directly*, with no projection and no merge. For world-changing interleavings, the statements below assume two explicit frame conditions: effects of participant-disjoint actions commute, and an action moved under a pending marker preserves that marker's formula. Communication-only reductions satisfy both conditions vacuously.

Lemma 3 (Residual capability)**.** *If $(G, W) \rightsquigarrow_\Lambda (G', W')$, then $\Lambda \in \mathsf{cap}(G)$.*

Lemma 4 (Participant agreement)**.** *If $G \vdash (\mathbb{M}; W)$, then $\mathsf{prt}(G) = \mathsf{prt}(\mathbb{M})$.*

Both follow by induction on the transition or typing derivation, respectively; they are the capability-membership and participant invariants used in the interleaving cases below.

Theorem 4 (Subject reduction)**.** *If $G \vdash (\mathbb{M}; W)$ and $(\mathbb{M}, W) \xrightarrow{\Lambda} (\mathbb{M}', W')$, then $(G, W) \rightsquigarrow_\Lambda (G', W')$ for some G' with $G' \vdash (\mathbb{M}'; W')$.*

Theorem 5 (Session fidelity)**.** *If $G \vdash (\mathbb{M}; W)$ and $(G, W) \rightsquigarrow_\Lambda (G', W')$, then $(\mathbb{M}, W) \xrightarrow{\Lambda} (\mathbb{M}', W')$ for some $\mathbb{M}'$ and W' with $G' \vdash (\mathbb{M}'; W')$.*

We prove both by induction on the typing derivation $G \vdash (\mathbb{M}; W)$, with a case analysis on the transition; the one auxiliary fact used throughout is that typing respects pointwise-equal sessions (`ctypes_ext`), which lets the interleaving cases reassociate independent updates.

Proof of Theorem 4 (subject reduction). By induction on $G \vdash (\mathbb{M}; W)$.

Case T-End ($G = \mathsf{End}$, every role idle). Then $\mathbb{M}$ has no output or pending action, so the only available transition is the goal observation S-Goal at $\Lambda = \checkmark\varphi_{goal}$ when $W \vDash \varphi_{goal}$, which leaves $(\mathbb{M}, W)$ fixed; there is no residual type to preserve and the claim holds (otherwise no Λ-transition exists and it holds vacuously).

Case T-Goal ($G = \checkmark\varphi.G_0$, with $W \vDash \varphi$ and $G_0 \vdash (\mathbb{M}; W)$, $\mathsf{prt}(G_0) = \mathsf{prt}(\mathbb{M})$). The transition $(\mathbb{M}, W) \to_\Lambda (\mathbb{M}', W')$ splits on its label.

- *Goal observation* ($\Lambda = \checkmark\varphi$, by S-Goal, so $\mathbb{M}' = \mathbb{M}$ and $W' = W$). Since $W \vDash \varphi$, rule G-Goal-E discharges the marker, $(\checkmark\varphi.G_0, W) \rightsquigarrow_{\checkmark\varphi} (G_0, W)$, and $G_0 \vdash (\mathbb{M}; W)$ is the premise: take $G' = G_0$.
- *Interleaving* ($\Lambda \neq \checkmark\varphi$). A goal marker is not a session construct, so this is already a transition of G_0's session; the induction hypothesis gives $(G_0, W) \rightsquigarrow_\Lambda (G', W')$ with $\Lambda \in \mathsf{cap}(G_0)$ and $G' \vdash (\mathbb{M}'; W')$. When the move is world-preserving (a communication, $W' = W$), G-Goal-I carries the still-pending marker along, $(\checkmark\varphi.G_0, W) \rightsquigarrow_\Lambda (\checkmark\varphi.G', W)$, and $\checkmark\varphi.G' \vdash (\mathbb{M}'; W)$ by T-Goal (the side condition $\mathsf{prt}(G') = \mathsf{prt}(\mathbb{M}')$ is preserved because Λ acts on the same participants on both sides). When the move is *world-changing* (a capability firing, $W' \neq W$), G-Goal-I still commutes it under the marker, $(\checkmark\varphi.G_0, W) \rightsquigarrow_\Lambda (\checkmark\varphi.G', W')$; re-typing the residual by T-Goal then additionally asks $W' \vDash \varphi$. This follows from the goal-stability hypothesis on actions commuting under pending markers. The global type sequences the marker before G_0's actions whereas the session may fire the action first; the interleaving rule lets the type catch up under that hypothesis. Removing the hypothesis, rather than this conditional case, is the open correspondence problem recorded in Section 7.

Case T-Comm, with head $\mathsf{p}{\to}\mathsf{q}$ and branches G_i. Here $\mathbb{M}$ is $\mathsf{p}[\mathsf{q}!\dots] \parallel \mathsf{q}[\mathsf{p}?\dots] \parallel \mathbb{M}_0$, and each branch satisfies $G_i \vdash (\mathsf{p}[P_i] \parallel \mathsf{q}[Q_i] \parallel \mathbb{M}_0; W)$. If the transition is a goal observation $\Lambda = \checkmark\varphi$ (S-Goal, leaving $(\mathbb{M}, W)$ fixed), it is matched by G-Comm-I: $\mathsf{prt}(\checkmark\varphi) = \emptyset$ makes the disjointness premise trivial, and each branch realises the same observation by the induction hypothesis, using $\checkmark\varphi \in \mathsf{cap}(G_i)$ — the observation concerns a marker the protocol actually pends (see the note below). Otherwise the transition is a communication $\Lambda = \mathsf{r}\,\ell\,\mathsf{t}$. Because p offers only outputs to q and q only inputs from p, the sender/receiver shapes force a dichotomy.

- *Head move* ($\mathsf{r} = \mathsf{p}$). Then $\mathsf{t} = \mathsf{q}$ and $\ell = \ell_k \in I$, and $\mathbb{M}' = \mathsf{p}[P_k] \parallel \mathsf{q}[Q_k] \parallel \mathbb{M}_0$. Rule G-Comm-E gives $(G, W) \rightsquigarrow_{\mathsf{p}\ell_k\mathsf{q}} (G_k, W)$, and $G_k \vdash (\mathbb{M}'; W)$ is exactly the k-th premise.
- *Interleaving move* ($\mathsf{r} \neq \mathsf{p}$). The shapes then force $\mathsf{r}, \mathsf{t} \notin \{\mathsf{p}, \mathsf{q}\}$: r is an output so $\mathsf{r} \neq \mathsf{q}$, and t is an input so $\mathsf{t} \neq \mathsf{p}$, while $\mathsf{t} = \mathsf{q}$ would force $\mathsf{r} = \mathsf{p}$. Since r, t lie in the bystanders $\mathbb{M}_0$, the *same* communication is enabled in each branch's configuration $\mathsf{p}[P_i] \parallel \mathsf{q}[Q_i] \parallel \mathbb{M}_0$ (the head updates do not touch r, t). The induction hypothesis at each i yields G_i' with $(G_i, W) \rightsquigarrow_\Lambda (G_i', W)$ and $G_i' \vdash (\mathsf{p}[P_i] \parallel \mathsf{q}[Q_i] \parallel \mathbb{M}_0'; W)$, where $\mathbb{M}_0'$ is $\mathbb{M}_0$ after the bystander step. As $\mathsf{prt}(\Lambda) = \{\mathsf{r}, \mathsf{t}\}$ is disjoint from $\{\mathsf{p}, \mathsf{q}\}$, rule G-Comm-I assembles $(G, W) \rightsquigarrow_\Lambda (\mathsf{p}{\to}\mathsf{q}{:}\{\ell_i.G_i'\}, W)$, and re-applying T-Comm (the sender/receiver at p, q are untouched, and each branch is retyped by the induction hypothesis, reassociating the independent updates via `ctypes_ext`) gives the residual typing.

Communications leave the world fixed, so no independence hypothesis is needed. For a world-changing action at the head (T-Act), G-Act-E matches and the residual G is typed at the *same* post-effect world the rule reaches — this is exactly where the lockstep world pairing of the corrected T-Act is used; a goal observation at an action-typed configuration is matched by G-Act-I exactly as in the T-Comm case, since $\mathsf{prt}(\checkmark\varphi) \cap \{\mathsf{p}\} = \emptyset$ trivially. Interleaving *two* actions requires their effects to commute ($\mathsf{eff}_a \circ \mathsf{eff}_b = \mathsf{eff}_b \circ \mathsf{eff}_a$ for disjoint a, b), a frame-independence side condition that participant-disjointness alone does not supply; under it the T-Act interleaving case goes through identically. $\square$

Proof of Theorem 5 (session fidelity). By induction on $G \vdash (\mathbb{M}; W)$, inverting the global step $(G, W) \rightsquigarrow_\Lambda (G', W')$. T-End admits only the goal observation G-Goal-E at $\checkmark\varphi_{goal}$, matched on the session by S-Goal. For T-Goal, the step is either G-Goal-E — matched by S-Goal on the session, the marker discharged — or G-Goal-I, whose premise $(G_0, W) \rightsquigarrow_\Lambda (G_0', W')$ the induction hypothesis realises as an underlying session step that carries the marker along. For T-Comm, the global step is either G-Comm-E — and the prescribed head communication $\mathsf{p}\,\ell_k\,\mathsf{q}$ is enabled by S-Comm, landing in the k-th premise — or G-Comm-I with $\mathsf{prt}(\Lambda) \cap \{\mathsf{p}, \mathsf{q}\} = \emptyset$; then the induction hypothesis on any branch produces a bystander communication, whose endpoints lie outside $\{\mathsf{p}, \mathsf{q}\}$, so S-Comm fires it on $\mathbb{M}$ and T-Comm retypes the result. The action cases mirror those of Theorem 4. $\square$

Mechanization. Both theorems are mechanized axiom-free for the single-label communication fragment — the fragment in which bystander interleaving is unconditional — in `DirectTypingSR.v`, over labelled session and global transition systems, with no projection, merge, or local type:

```
Theorem subject_reduction :  forall W G s s' L,
ctypes W G s -> lstep L s s' ->
exists G', gstep W L G G' /\ ctypes W G' s'.
Theorem session_fidelity :  forall W G G' s L,
ctypes W G s -> gstep W L G G' ->
exists s', lstep L s s' /\ ctypes W G' s'.
(* Print Assumptions => Closed under the global context (both).  *)
```

The head-move case for world-changing *actions* is additionally mechanized in `DirectTyping.v` (`type_directed_safety`, also `progress`). Observable goal labels appear in neither Coq development: the mechanized label alphabet is communication-only, and its goal rule fuses marker discharge with a continuation step. Participant-matching side conditions, labelled action interleaving, and multi-branch choice are likewise absent. Completing a faithful mechanization of the paper rules remains work in Section 7.

Concrete instance. `HandoffInstance` mechanizes the planner/worker handoff of Section 1 on both sides. The *good* pairing — planner sends, worker delivers, goal checked — is proved `ctypes`-typed (`good_typed`) and its run is proved to reach a world satisfying the goal (`good_runs_to_goal`). The *bad* pairing — both roles start with an input, exactly the deadlock of Section 1 — is proved `Stuck` (`bad_stuck`), and the contrapositive of `progress` then gives, for free, that *no non-trivial global type can ever type it* (`bad_untypeable`): the rule rejects the classical deadlocking handoff without any projection ever being computed.

D.4 Incompleteness, by design

Proposition 1 (Incompleteness)**.** *$\Gamma; G \vDash \Diamond_{init}\varphi_{goal}$ may hold while no concrete runtime execution reaches φ_{goal}: the witnessing abstract path can be spurious when α collapses a distinction that mattered. The system is* sound for ✗ and incomplete for ✓.

This is the price of decidable, tolerant over-approximation; tightening α trades tolerance for precision. The residue is confined to two edges, motivating the layered architecture:

- **Payload faithfulness** (bottom edge): a tool's declared post-condition ("filters to fares < 500") may be false — a runtime obligation for a deterministic monitor (Layer C), not a static one.
- **Intent fidelity** (top edge): the formalized goal may not capture what the user meant — irreducibly semantic, surfaced at the single compaction checkpoint for human review.

D.5 Decidability and the autonomy boundary

Theorem 6 (Decidability)**.** *In the fragment with finitely many roles (static topology), regular global types represented by finite tail-recursive control graphs, decidable state logic $\mathcal{L}$, and $\mathcal{L}$-definable effects, $\Gamma; G \vDash \Diamond_{init}\varphi_{goal}$ is decidable.*

Proof. The pack-level achievability judgment $\Gamma; G \vDash \Diamond_{init}\varphi_{goal}$ (Section B.2) is an existential reachability query over configurations (G', W); we exhibit a terminating decision procedure and argue its completeness.

The search space is finite. A configuration has two components. (i) *Control*, the residual global type G' reachable from G by $\rightsquigarrow$. Since G is a regular tree (tail-recursion: finitely many distinct subtrees, Section A.3) and each $\rightsquigarrow$ step either descends into a subtree (G-COMM-E, G-ACT-E) or erases a marker (G-GOAL-E), the set $\{G' : (G, \_) \rightsquigarrow^* (G', \_)\}$ is a subset of the finitely many subtrees of G, hence finite. (ii) *Symbolic world.* The boolean part is a valuation $B : \mathsf{Pred} \to \mathbb{B}$ over the finitely many predicates named in the pack, so there are $2^{|\mathsf{Pred}|}$ boolean states. The numeric part is summarized by an accumulated constraint $\psi \in \mathcal{L}$ (a conjunction of the guards taken and the Nd-postconditions assumed); on a control-graph back edge the reference procedure applies the widening $\psi \mapsto \top$, forgetting accumulated numeric constraints. This one-element numeric summary is an over-approximation and, together with the finite boolean valuations, leaves only finitely many symbolic worlds. The product with finite control is therefore finite.

Each edge is computable. Firing a capability requires checking $W \vDash pre_a$ and forming sp; testing goal-satisfaction requires $W \vDash \varphi$. Both are entailment/satisfiability queries in quantifier-free linear integer arithmetic with uninterpreted predicates — a decidable theory, discharged by an SMT solver. Branch selection (REACH-OR) is a finite disjunction. Existence of an initial world is decided by the same SMT query over $init$; each satisfying symbolic initial valuation seeds the finite search.

Termination and correctness. Forward search (breadth-first over $\rightsquigarrow$) maintains a visited set of (control, symbolic-world) pairs; because that set is finite it adds each pair at most once, so the search halts. It answers ACHIEVABLE iff it enumerates a goal-satisfying configuration, and IMPOSSIBLE otherwise. Soundness of the IMPOSSIBLE verdict is Theorem 1; and widening only *enlarges* the reachable set (it weakens constraints, adding edges), so by Theorem 2 it never introduces a false refutation. Hence the procedure is a decision procedure for $\Gamma; G \vDash \Diamond_{init}\varphi_{goal}$. ☐

Proposition 2 (Undecidability in a dynamic-topology extension)**.** *Consider the extension G^+ of the grammar with* `spawn r.` *G, which creates fresh participants at run time. With an unbounded role population and dynamic channels, achievability in G^+ is undecidable.*

Proof sketch. By reduction from the control-state reachability problem for communicating finite-state machines (CFSMs), which is undecidable [15]. Fix a CFSM instance: finite-control machines M_1, M_2 exchanging messages over unbounded FIFO channels, and a target control state q_f of M_1; deciding whether a run reaches q_f is undecidable.

We compute, from this instance, a skill whose goal is achievable iff the CFSM reaches q_f. Each machine M_i becomes a role whose process mirrors its control graph: a transition that sends message m spawns, at run time, a fresh *courier* participant carrying m (this is exactly the dynamic-spawning capability the theorem hypothesizes), and a transition that receives m synchronizes with the oldest outstanding courier for that channel, so the unbounded family of couriers realizes the unbounded FIFO. The required order can be made explicit by assigning sequence numbers through the numeric world state and guarding receives on the next sequence number; the Asg/Nd machinery of Section A.2 supplies those updates. A boolean predicate $\mathtt{at\_}q_f$ is established by the single capability guarding M_1's entry into q_f, and we set $\varphi_{goal} = \mathtt{at\_}q_f$. By construction a run of the skill reaches a state satisfying φ_{goal} iff the CFSM has a run reaching q_f: the courier discipline then puts the skill's reachable configurations in correspondence with the CFSM's channel contents and control states. The construction is a computable map from CFSM instances to packs, so a decision procedure for achievability would decide CFSM control-state reachability — impossible. Hence achievability is undecidable once participants may be spawned dynamically. (The finiteness argument of Theorem 6 breaks at exactly the point this reduction exploits: an unbounded number of couriers makes the set of reachable controls infinite.) ☐

The two results isolate one concrete boundary: static finite topology admits a total decision procedure, while unbounded dynamic spawning does not. This is a boundary on topology, not a claim that all forms of agent autonomy are undecidable. The implementation returns UNKNOWN when a pack violates the static-topology side condition of Theorem 6, unless an independent capability or establisher refutation has already proved it impossible.

D.6 The partition of obligations

Corollary 1. *The architecture separates "will the goal be achieved" into four obligations on disjoint substrates:*

Concern	Where	Mechanism
goal achievability	static, decidable	Coq specification; deterministic search; no LLM
per-step safety / least privilege	Layer B (future)	same engine, $\Box$ + permission tiers
payload faithfulness	Layer C (future), runtime	deterministic monitors with blame
intent fidelity	top edge, synthesis	one human checkpoint

D.7 Verifiable feedback beyond a scalar reward

The verdict is a heterogeneous verification signal rather than a scalar score. An IMPOSSIBLE result identifies which premise failed and carries a blocking frontier; ACHIEVABLE carries a witness path and explicitly retains the two deferred obligations; UNKNOWN is an abstention outside the decidable fragment, not a refutation. These structured outcomes can serve as cheap negative labels for automated skill or prompt search without an LLM judge, while the intent-fidelity checkpoint keeps the irreducible semantic decision with a human. We treat such search as an application of the verifier, not as a contribution evaluated here.

E Token economics of pre-execution refutation

The practical case for deciding achievability *before* execution is an economic one, so we state it in tokens rather than adjectives. Two asymmetries carry it. First, no model sits in the trusted path: the schema gate, projection, conformance and Z3 reachability spend **zero tokens**, and the deterministic front-end spends zero as well. Only the optional LLM compaction spends tokens, and it is paid *once per skill version*. Second, an agent turn re-sends its whole context: writing S for the harness prompt, K for the loaded skill, g for the mean tokens appended per turn and o for output per turn, a run of T turns reads $T(S{+}K) + g\,T(T{-}1)/2$ input and To output tokens. Compaction is linear in the skill and paid once; a doomed run is *quadratic* in its turns and recurs on every invocation.

Runtime waste is necessarily a model — it prices a run that, if the refutation is correct, never happens — so we publish the model rather than a single number. Each refutation reason carries a (low, typical, high) band for how long an agent flails before that failure stops it, and how many agents are billed. With conservative defaults ($S{=}2500$, $g{=}700$, $o{=}250$) and a median real consumer skill ($K \approx 1.7$K tokens), compaction costs $\sim$2.8K tokens against the following avoided waste per prevented invocation:

Verdict reason	turns (lo–typ–hi)	waste/run	leverage
MISSING_CAPABILITY	3–8–14, 1 agent	50 240	18×
GOAL_UNSAT	8–18–30, 1 agent	176 040	63×
NON_CONFORMANT	6–15–30, 2 agents	261 900	94×
NON_PROJECTABLE	6–15–30, 2 agents	261 900	94×
BLOCKED_GUARD	12–25–50, 1 agent	305 750	110×

The ordering is the robust part, and it is the expected one. MISSING_CAPABILITY is cheapest at run time: the agent calls a tool that is not there and learns immediately, and most of the cost is the improvisation that follows. BLOCKED_GUARD is dearest, because a precondition no run can satisfy is precisely the case an agent cannot learn from — it retries to the harness turn cap. The two multi-party reasons bill two contexts. GOAL_UNSAT additionally carries a cost the token bill does not show: a run whose goal conjunct has no establisher can terminate *believing* it succeeded. DYNAMIC_TOPOLOGY claims no savings at all, because an abstention prevents nothing.

Over the 15-spec corpus the seven structural refutations cost 12.2K tokens of compaction and avoid $\sim$1.07M tokens per round of invocations (band 0.28M–3.15M): $\sim$87$\times$ leverage, or nothing at all on the deterministic path. Break-even therefore falls *inside* the first prevented run in every mode — between 0.9% and 5.5% of it. The honest denominator for a healthy skill, where the check buys nothing and still costs something, is 2.9% of one successful run over the corpus and 4.0% for a median real skill; with the deterministic front-end, zero. Prompt caching changes the price of the wasted tokens, not their number. Halving every waste default leaves leverage at $9\times$–$55\times$ and the ordering unchanged. The model, its defaults and their justification are in the artifact (`skillc.tokens`, `skillc cost`); the Coq development costs no tokens and is amortized over every skill ever checked.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state that refutation is sound relative to the declared pack, distinguish UNKNOWN from refutation, and identify the mechanized and paper-only proof fragments.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The Limitations and Future Work section discusses relative soundness, static topology, abstraction coarseness, mechanization gaps, and the proof-of-concept corpus.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: Appendix D states the simulation, stability, commutativity, finiteness, and topology assumptions with proofs or proof sketches; the artifact checks the explicitly identified Coq fragments.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Sections 4–5 describe the deterministic checker, verdict semantics, corpus construction, binary accounting, and separate treatment of abstentions.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The anonymized supplemental artifact contains the checker, synthetic corpora, proof developments, pinned continuous-integration workflow, and reproduction commands.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [N/A]

Justification: The evaluation is a deterministic execution over declared packs; it performs no training, optimization, sampling, or data splitting.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [N/A]

   Justification: Each corpus case and checker result is deterministic, so repeated trials have no sampling variance; the paper reports exact counts rather than inferential statistics.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Section 5 states that evaluation is CPU-only, requires no model inference or training, and gives the corpus size and solver timeout governing the run.

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses synthetic packs, no personal data or human subjects, and makes bounded claims with explicit trust assumptions and limitations.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The Broader impact paragraph discusses reduced wasted execution as well as false confidence from inaccurate packs and the human/runtime mitigations required.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The artifact releases no trained model, scraped dataset, or other high-risk asset; it contains a deterministic checker and synthetic examples.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [N/A]

    Justification: The evaluation does not reuse an external dataset or model; prior methods and software foundations are credited through citations.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The supplemental artifact documents the checker interface, pack schema, synthetic corpora, proof scope, limitations, and reproduction workflow.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The research involves neither crowdsourcing nor human-subject experiments.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The research involves no study participants or human-subject data.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: Sections 1 and 4 describe LLM compaction and bounded repair, explicitly place them outside the trusted checker, and state that the reported corpus evaluation does not depend on live model inference.